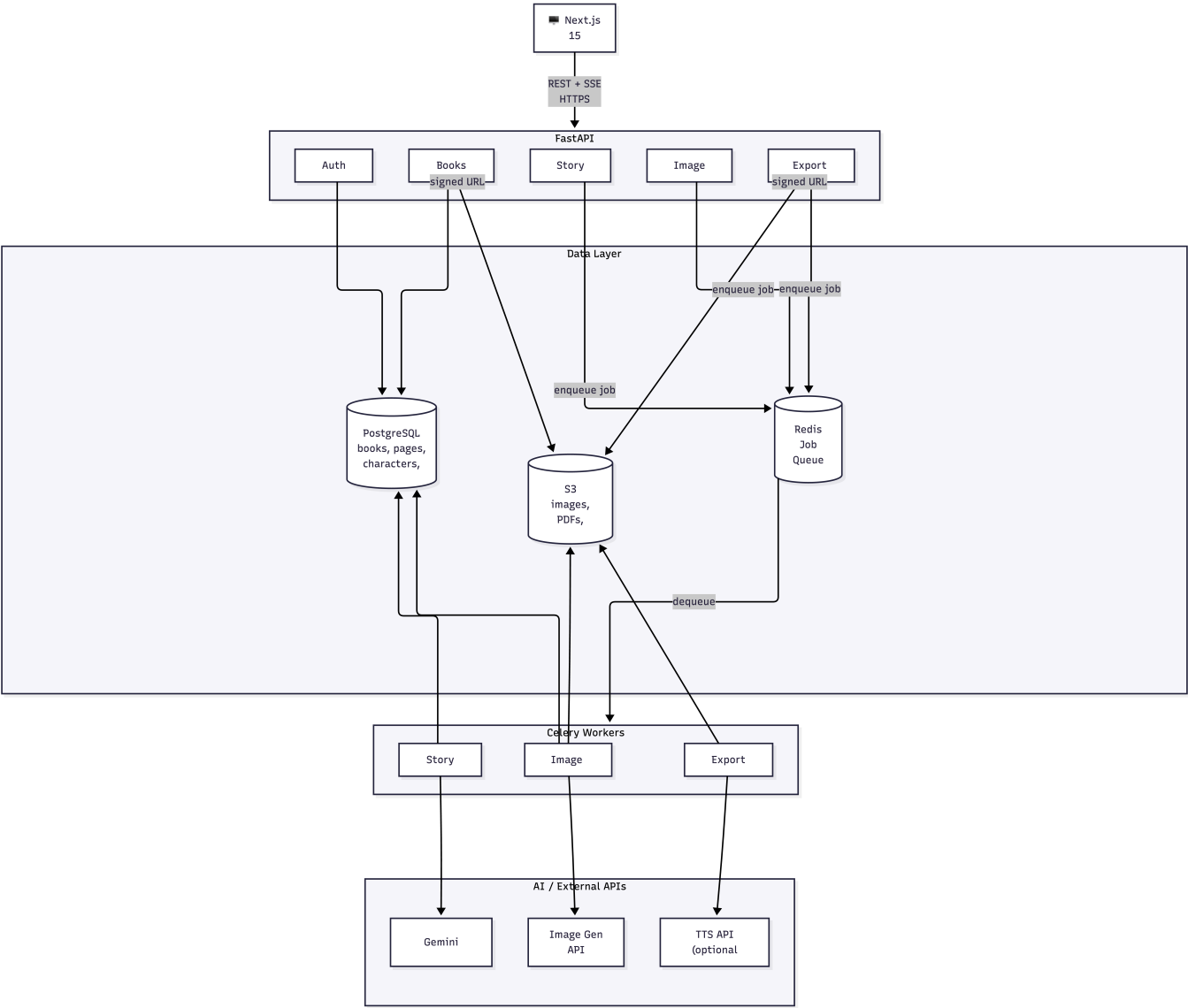
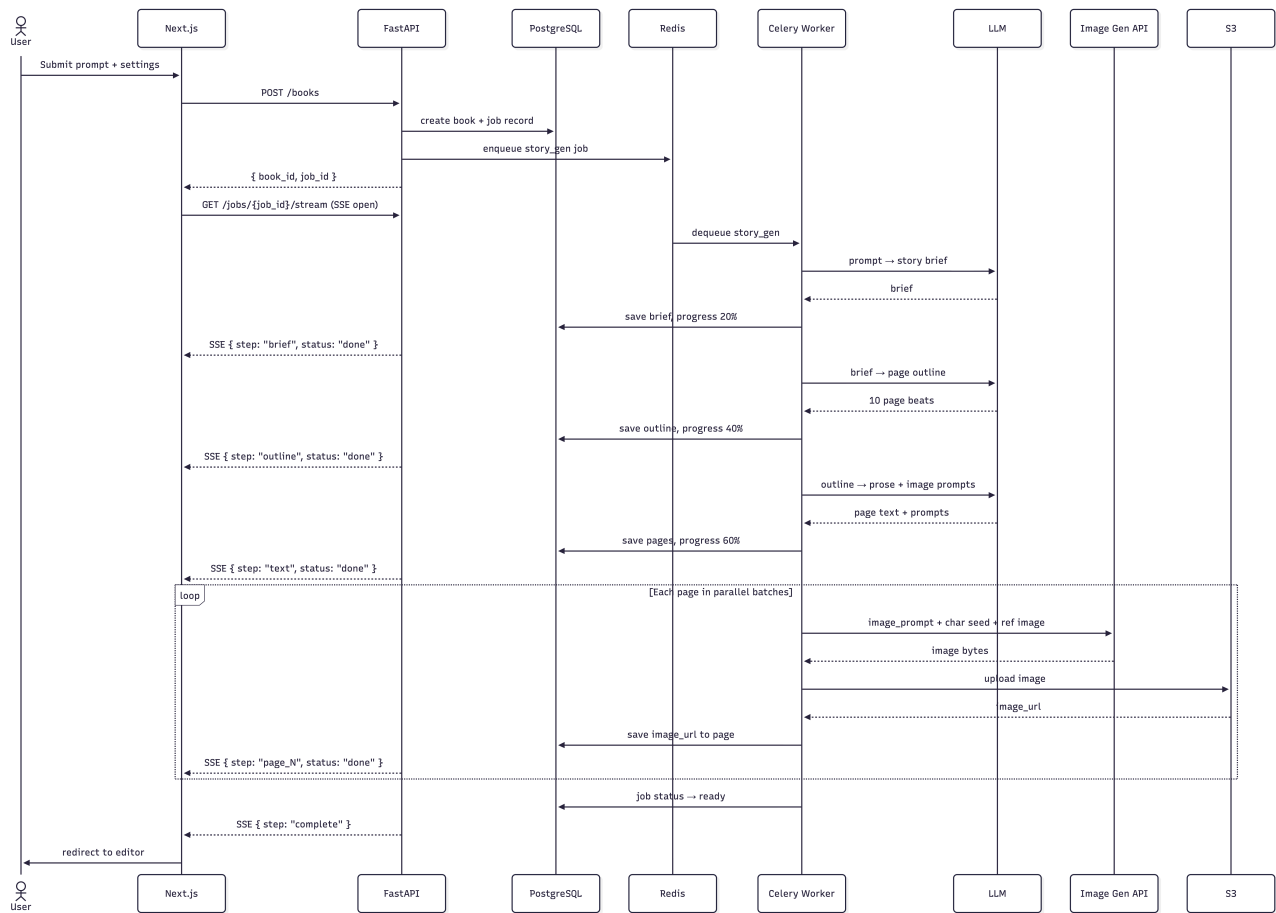


AI Storybook Studio - Architecture Doc

High Level Sys Arch



Flow



Backend Tech Stack

Component	Local	AWS
API	FastAPI (local)	ECS Fargate
Queue	RabbitMQ	SQS
Cache	Redis	ElastiCache Redis
Workers	Celery + RabbitMQ	Celery + SQS / Lambda
Database	PostgreSQL	RDS PostgreSQL
Storage	MinIO	S3
CDN	—	CloudFront
Secrets	.env	Secrets Manager

API Contract

Auth

Method	Endpoint	Body	Notes
POST	/auth/register	email, password, pen_name	Returns JWT
POST	/auth/login	email, password	Returns JWT
POST	/auth/google	id_token	OAuth2 flow
POST	/auth/refresh	refresh_token	Rotate JWT
POST	/auth/logout	—	Blacklist token

Books (the project object)

Method	Endpoint	Body / Params	Notes
POST	/books	prompt, age_range, tone[], art_style, safety_on	Creates book record, status: draft
GET	/books	?status=&page=&limit=	Library — paginated
GET	/books/{id}	—	Full book with pages + characters
PATCH	/books/{id}	title, age_range, tone, art_style, safety_on	Edit book config
DELETE	/books/{id}	—	Soft delete
PATCH	/books/{id}/visibility	is_public: bool	Toggle shareable link

Generation (Async — all push to queue)

Method	Endpoint	Body	Notes
POST	/books/{id}/generate/brief	—	Step 1: prompt → story brief
POST	/books/{id}/generate/outline	—	Step 2: brief → page beats + characters
POST	/books/{id}/illustrate	—	Step 3: all pages → images (batched)
GET	/jobs/{job_id}	—	Poll job status + progress %

Method	Endpoint	Body	Notes
GET	/jobs/{job_id}/stream	—	SSE — real-time progress events

SSE event shape:

```
{ "step": "page_3", "status": "done", "progress": 60 }
{ "step": "complete", "status": "done", "redirect": "/editor" }
{ "step": "page_5", "status": "error", "message": "... " }
```

Outline

Your instinct to keep outline edits separate is right. Here's how to avoid the "too many options" problem — **one endpoint per action**, not one endpoint with a giant action field.

Method	Endpoint	Body	Notes
GET	/books/{id}/outline	—	All beats in order
PATCH	/books/{id}/outline/pages/{page_id}	text	Edit beat text directly
POST	/books/{id}/outline/pages/{page_id}/regenerate	instruction?	Async - regenerate with optional hint
POST	/books/{id}/outline/pages	text, position	Add new page at position
DELETE	/books/{id}/outline/pages/{page_id}	—	Remove beat
PATCH	/books/{id}/outline/pages/reorder	order: [id, id, ...]	Drag-and-drop reorder

Characters

Method	Endpoint	Body	Notes
GET	/books/{id}/characters	—	All character with traits
POST	/books/{id}/characters	name, description, traits[]	Add character
PATCH	/books/{id}/characters/{char_id}	name, traits[]	Edit traits
POST	/books/{id}/characters/{char_id}/regenerate	instruction?	Async — new reference image
DELETE	/books/{id}/characters/{char_id}	—	Remove character

Pages / Editor

Method	Endpoint	Body	Notes
GET	/books/{id}/pages	—	All pages with image URLs
PATCH	/books/{id}/pages/{page_id}/text	text	Edit text directly (sync)
PATCH	/books/{id}/pages/{page_id}/prompt	image_prompt	Update image prompt
POST	/books/{id}/pages/{page_id}/regenerate/image	image_prompt?	Async — new illustration
POST	/books/{id}/pages/{page_id}/regenerate/text	instruction?	Async — Claude rewrites text

Reader

Method	Endpoint	Notes
GET	/books/{id}/reader	Auth-required, private book
GET	/books/public/{share_token}/reader	No auth, public books only

Response shape:

```
{
  "id": "uuid",
  "title": "The Brave Little Fox",
  "style": "watercolor",
  "age_range": "3-5",
  "pages": [
    {
      "page_number": 0,
      "is_cover": true,
      "text": "The Brave Little Fox",
      "image_url": "https://signed-s3-url...expires=3600"
    }
  ]
}
```

Image URLs are **signed S3/GCS URLs** (1hr TTL) — API never proxies binary data.

Export

Method	Endpoint	Body	Notes
POST	/books/{id}/export/pdf	—	Async — queues PDF build
POST	/books/{id}/export/epub	—	Async — queues EPUB build
GET	/books/{id}/exports	—	List past exports
GET	/books/{id}/exports/{export_id}/download	—	Returns signed download URL

Notifications

Method	Endpoint	Notes
GET	/notifications	List unread notifications
PATCH	/notifications/{id}/read	Mark one as read
PATCH	/notifications/read-all	Mark all read

Notifications are written to DB by workers when async jobs complete. SSE handles real-time delivery while the user is on the page. Notifications table handles persistence for when they're not.

Database Design

Tables

users

```
CREATE TABLE users (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email       TEXT UNIQUE NOT NULL,  
  password_hash TEXT,                                -- NULL for OAuth-only  
  pen_name    TEXT NOT NULL,  
  avatar_url  TEXT,  
  created_at  TIMESTAMPTZ DEFAULT now(),  
  updated_at  TIMESTAMPTZ DEFAULT now()  
);
```

oauth_accounts

```
CREATE TABLE oauth_accounts (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  provider    TEXT NOT NULL,                        -- 'google'  
  provider_user_id TEXT NOT NULL,  
  access_token TEXT,  
  refresh_token TEXT,  
  expires_at  TIMESTAMPTZ,
```

```
    UNIQUE (provider, provider_user_id)
);
```

refresh_tokens

```
CREATE TABLE refresh_tokens (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash  TEXT NOT NULL UNIQUE,
  expires_at  TIMESTAMPTZ NOT NULL,
  revoked_at  TIMESTAMPTZ          -- NULL = still valid
);
```

books

The root project object everything hangs off.

```
CREATE TYPE book_status AS ENUM (
  'draft',          -- created, params saved, nothing generated yet
  'processing',     -- an async job is currently running
  'outline_ready',  -- outline + characters generated, awaiting
  illustration
  'illustrating',   -- image generation in progress
  'ready',          -- fully illustrated, can export
  'error'           -- last job failed, check jobs table
);

CREATE TABLE books (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title       TEXT NOT NULL DEFAULT 'Untitled',
  prompt      TEXT NOT NULL,          -- original user prompt
  pen_name    TEXT,                  -- overrides user pen_name
  if set
  age_range   TEXT NOT NULL,         -- '3-5' | '6-8' | '9-11'
  tone        TEXT[] NOT NULL DEFAULT '{}', -- ['Funny', 'Cozy']
  art_style   TEXT NOT NULL,         -- 'watercolor' | 'crayon'
  etc
  safety_on   BOOLEAN NOT NULL DEFAULT true,
  status      book_status NOT NULL DEFAULT 'draft',
```



```
is_public      BOOLEAN NOT NULL DEFAULT false,  
share_token    TEXT UNIQUE DEFAULT gen_random_uuid()::TEXT,  
created_at     TIMESTAMPTZ DEFAULT now(),  
updated_at     TIMESTAMPTZ DEFAULT now()  
);
```

story_brief

One-to-one with `books`. Stores the structured brief generated from the prompt before outline generation.

```
CREATE TABLE story_brief (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  book_id     UUID NOT NULL UNIQUE REFERENCES books(id) ON DELETE  
CASCADE,  
  title       TEXT,  
  setting     TEXT,  
  main_character TEXT,  
  lesson      TEXT,  
  synopsis    TEXT,  
  raw_json    JSONB,                -- full LLM response, for  
debugging  
  created_at  TIMESTAMPTZ DEFAULT now(),  
  updated_at  TIMESTAMPTZ DEFAULT now()  
);
```

characters

```
CREATE TABLE characters (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  book_id     UUID NOT NULL REFERENCES books(id) ON DELETE  
CASCADE,  
  name        TEXT NOT NULL,  
  description  TEXT,  
  traits      TEXT[] NOT NULL DEFAULT '{}',  
  reference_image_url TEXT,        -- S3 key, locked reference  
sheet  
  image_seed  INTEGER,            -- fixed seed for  
consistency across pages  
  is_locked   BOOLEAN NOT NULL DEFAULT false,
```

```
    created_at          TIMESTAMPTZ DEFAULT now(),
    updated_at          TIMESTAMPTZ DEFAULT now()
);
```

pages

Covers both outline stage (only `outline_text`) and illustrated stage (`text` + `image_url`). Same row evolves through the pipeline.

```
CREATE TYPE page_status AS ENUM (
    'outline',      -- beat text only, no image yet
    'generating',   -- image/text being generated
    'ready',        -- fully illustrated
    'error'
);

CREATE TABLE pages (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    book_id           UUID NOT NULL REFERENCES books(id) ON DELETE CASCADE,
    page_number       INTEGER NOT NULL,
    is_cover          BOOLEAN NOT NULL DEFAULT false,
    outline_text      TEXT,                                -- short beat summary from
outline stage
    text              TEXT,                                -- final prose for the page
    image_url         TEXT,                                -- S3 key
    image_prompt      TEXT,                                -- stored for regeneration
    image_seed        INTEGER,                             -- stored for consistency
    status            page_status NOT NULL DEFAULT 'outline',
    created_at        TIMESTAMPTZ DEFAULT now(),
    updated_at        TIMESTAMPTZ DEFAULT now(),
    UNIQUE (book_id, page_number)
);
```

page_characters

Junction table — which characters appear on which page. Drives targeted consistency checks during image regeneration.

```
CREATE TABLE page_characters (
    page_id           UUID NOT NULL REFERENCES pages(id) ON DELETE CASCADE,
```

```
character_id UUID NOT NULL REFERENCES characters(id) ON DELETE CASCADE,  
PRIMARY KEY (page_id, character_id)  
);
```

jobs

Tracks every async operation. `page_id` and `character_id` are nullable — only set when the job is scoped to a specific page or character (e.g. single page regeneration).

```
CREATE TYPE job_type AS ENUM (  
    'brief_gen',          -- prompt → story brief  
    'outline_gen',        -- brief → outline + characters  
    'illustration',       -- full book illustration  
    'page_regen_image',   -- single page image regeneration  
    'page_regen_text',    -- single page text regeneration  
    'char_regen',         -- character reference image regeneration  
    'pdf_export',  
    'epub_export'  
);  
  
CREATE TYPE job_status AS ENUM (  
    'pending',  
    'running',  
    'done',  
    'error'  
);  
  
CREATE TABLE jobs (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    book_id UUID NOT NULL REFERENCES books(id) ON DELETE CASCADE,  
    page_id UUID REFERENCES pages(id) ON DELETE SET NULL,  
    character_id UUID REFERENCES characters(id) ON DELETE SET NULL,  
    type job_type NOT NULL,  
    status job_status NOT NULL DEFAULT 'pending',  
    progress INTEGER NOT NULL DEFAULT 0 CHECK (progress BETWEEN 0 AND  
100),  
    error TEXT,  
    worker_id TEXT, -- Celery worker hostname  
    created_at TIMESTAMPTZ DEFAULT now(),  
    started_at TIMESTAMPTZ,  
    completed_at TIMESTAMPTZ  
);
```

notifications

Written by workers when jobs complete. SSE delivers them in real time while the user is active; this table persists them for when they return.

```
CREATE TYPE notification_type AS ENUM (  
    'outline_ready',  
    'illustration_ready',  
    'page_ready',  
    'export_ready',  
    'error'  
);  
  
CREATE TABLE notifications (  
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
    book_id     UUID REFERENCES books(id) ON DELETE SET NULL,  
    job_id      UUID REFERENCES jobs(id) ON DELETE SET NULL,  
    type        notification_type NOT NULL,  
    message     TEXT NOT NULL,  
    is_read     BOOLEAN NOT NULL DEFAULT false,  
    created_at  TIMESTAMPTZ DEFAULT now()  
);
```

exports

```
CREATE TYPE export_format AS ENUM ('pdf', 'epub');  
CREATE TYPE export_status AS ENUM ('pending', 'generating', 'ready',  
    'error');  
  
CREATE TABLE exports (  
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    book_id     UUID NOT NULL REFERENCES books(id) ON DELETE  
CASCADE,  
    format      export_format NOT NULL,  
    s3_key      TEXT,  
    file_size_bytes INTEGER,  
    status      export_status NOT NULL DEFAULT 'pending',  
    download_url TEXT,          -- cached signed URL  
    download_url_expires_at TIMESTAMPTZ, -- regenerate if past  
    expiry
```

```
        created_at          TIMESTAMPTZ DEFAULT now(),
        completed_at       TIMESTAMPTZ
    );
```

Indexes

```
-- Auth
CREATE INDEX ON oauth_accounts(user_id);
CREATE INDEX ON refresh_tokens(user_id);
CREATE INDEX ON refresh_tokens(token_hash);

-- Books
CREATE INDEX ON books(user_id);
CREATE INDEX ON books(status);
CREATE INDEX ON books(share_token) WHERE is_public = true;

-- Pages
CREATE INDEX ON pages(book_id, page_number);
CREATE INDEX ON pages(book_id, status);

-- Characters
CREATE INDEX ON characters(book_id);

-- Page characters
CREATE INDEX ON page_characters(character_id);

-- Jobs
CREATE INDEX ON jobs(book_id, status);
CREATE INDEX ON jobs(page_id) WHERE page_id IS NOT NULL;
CREATE INDEX ON jobs(status) WHERE status IN ('pending', 'running');

-- Notifications
CREATE INDEX ON notifications(user_id, is_read);
CREATE INDEX ON notifications(user_id, created_at DESC);

-- Exports
CREATE INDEX ON exports(book_id, format);
```
